

Claude Code Plugins Research Report

Core conclusion

The strongest framing for your newsletter is sound: **Claude Code plugins are not primarily a new capability layer; they are the packaging, distribution, and lifecycle layer for Claude Code customisation.** Anthropic's own "Extend Claude Code" guide says exactly that in functional terms: CLAUDE.md, skills, MCP, subagents, hooks, and code intelligence solve different extension problems, while **plugins and marketplaces package and distribute those features.** Anthropic's plugin docs then define a plugin as a **self-contained directory of components** and explicitly position plugins as the choice when you want to share extensions across projects, version them, update them, and distribute them through a marketplace. ¹

That makes plugins a **moderately advanced** Claude Code topic. The filesystem mechanics are fairly approachable: Anthropic documents a quickstart that is basically "make a directory, add `.claude-plugin/plugin.json`, add a skill, test locally". The genuinely advanced part is operational: version policy, marketplace design, dependency constraints, update behaviour, trust and security, CI validation, ownership, and how to stop a neat helper from becoming a brittle little empire. That split makes plugins **quite suitable for a newsletter post**, provided the angle is not "how to install from a marketplace" but rather **"when personal Claude customisation becomes team infrastructure"**. ²

What Claude Code plugins are

Anthropic's reference defines a Claude Code plugin as a **self-contained directory of components that extends Claude Code with custom functionality.** The docs foreground six core component families: **skills, agents, hooks, MCP servers, LSP servers, and monitors.** The manifest schema and file-location reference also document additional packaged assets such as **commands, output styles, themes, executables in `bin/`, default `settings.json`, user-configurable options, and channels bound to MCP servers.** In other words, a plugin is broader than "a bundle of skills", but it is still mostly a **container around existing extension primitives**, not an entirely separate programming model. ³

Anthropic's own overview page makes the role especially clear: CLAUDE.md adds persistent context, skills add reusable workflows and knowledge, MCP connects external systems, hooks automate lifecycle events, subagents isolate work, and **plugins are the packaging layer** that bundles these pieces into a single installable unit. That is the cleanest official answer to "do plugins add a new capability?": **sometimes indirectly, yes, because they can package more than one primitive together; but conceptually they mostly standardise, namespace, distribute, and update capabilities that Claude Code already knows how to load.** ⁴

A plugin is discovered or installed in three main ways. First, Anthropic documents **marketplace installs**, where users browse `/plugin`, install from a known marketplace, and enable the plugin at user, project, local, or managed scope. Secondly, Anthropic documents **skills-directory plugins**, where any folder under a skills directory containing a `.claude-plugin/plugin.json` manifest is auto-loaded as `<name>@skills-dir` with no marketplace step. Thirdly, Anthropic documents **local development loading** through `--plugin-dir`, which is the intended path for development and testing. This is

important editorially: the plugin system spans the whole path from **local experimentation** to **shared infrastructure** rather than being only a public-marketplace feature. ⁵

The marketplace concept is official and concrete. Anthropic defines a plugin marketplace as a **catalogue** that provides **centralised discovery, version tracking, automatic updates, and support for multiple source types**. In the current docs, users can add marketplaces from GitHub shorthand, generic git URLs, local paths, or direct remote URLs to `marketplace.json`; inside a marketplace, individual plugins can come from relative paths, GitHub repositories, git repositories, git subdirectories in monorepos, or npm packages. That makes a marketplace closer to an internal package registry than to a simple list of prompt files. ⁶

After installation, a few things happen that matter for readers. Plugin skills are namespaced, so a plugin named `commit-commands` exposes things like `/commit-commands:commit`; the plugin is copied into the local versioned plugin cache; the `/plugin` interface can show what the plugin adds, including commands, agents, hooks, MCP/LSP servers, context cost, and last-updated metadata; and if installation happens mid-session, Anthropic recommends running `/reload-plugins` to activate the change without restarting. ⁷

Why plugins exist

Anthropic's official guidance on "when to use plugins vs standalone configuration" is the crisp answer here. Use standalone `.claude/` configuration for **single-project customisation, quick experiments, or personal workflows**. Use plugins when you need to **share functionality with your team or community, reuse the same setup across multiple projects, version and update extensions, or distribute them through a marketplace**. Anthropic's "Extend Claude Code" overview says the same thing more generally: you add CLAUDE.md, skills, hooks, MCP and subagents as needed; **once another repository needs the same setup, package it as a plugin**. ⁸

That is why plugins exist at all. Without them, team reuse tends to look like copying folders, copying snippets into `.claude/`, or asking every developer to "just install these three things and trust me". The docs do not dramatise those pains in so many words, but the features they do provide map directly to them: marketplace catalogues centralise discovery; scopes and managed settings standardise rollout; pinned sources and versions reduce drift; project scope and team marketplaces support shared onboarding; and auto-update plus explicit versions give maintainers controlled propagation paths. Put plainly, plugins solve the boring but essential problems of **repeatability, ownership, and change management**. ⁹

So the clean distinction is this: **skills, hooks, agents, and MCP are productivity primitives; plugins are infrastructure for distributing those primitives**. That is the angle your audience is likely to find useful, especially technical managers and ML leads who are already past "how do I write a prompt file?" and more interested in "how do I make this workflow stable across six repos and thirty people?". Anthropic's own migration guidance from existing `.claude/` content to plugin structure reinforces the same progression: start locally, then convert for easier sharing and distribution. ¹⁰

Installing from a marketplace is useful when the workflow is already standardised and somebody else is carrying the maintenance burden: official LSP plugins, external integrations such as GitHub, Slack, Vercel, or Sentry, and mature workflow plugins like `commit-commands` or `pr-review-toolkit` are obvious examples in Anthropic's catalogue. Building an internal plugin is more useful when the value comes from **your team's conventions** rather than generic tooling: how you review experiments, how you investigate incidents, how you write release notes, how you query your data platform, or how you

enforce repo-specific operational rules. Anthropic's docs on internal marketplaces and project-level `extraKnownMarketplaces` are clearly designed for that second use case. ¹¹

How plugins differ from the other Claude Code layers

Plugins and CLAUDE.md

CLAUDE.md is **always-on project context**. Anthropic says it loads every session and is best for conventions Claude should always know, such as build commands, architecture notes, or "never do X" rules. A plugin is not that. In fact, Anthropic's plugin reference explicitly says that a `CLAUDE.md` file placed at a plugin root is **not loaded as project context**; if you want instructions shipped through a plugin to load into Claude's context, the docs say to put them in a **skill** instead. Use CLAUDE.md when the information belongs to a specific repo and should be present in every conversation. Use a plugin when the same behaviour or workflow needs to travel across repos and installations. ¹²

Plugins and rules

Rules are still part of the always-on instruction layer, just more scoped. Anthropic positions `.claude/rules/` as a way to keep CLAUDE.md focused by loading file-path-specific guidance only when relevant. Rules solve **instruction scoping inside a project**. Plugins solve **distribution and packaging**. If you have a TypeScript rule that only matters in one monorepo, keep it as a rule. If your organisation wants the same audit workflow, review agents, and helper commands across many repos, that is when you move up a layer to a plugin. ⁴

Plugins and skills

Skills are the most important comparison. Anthropic describes skills as reusable knowledge and invocable workflows; they are markdown-based, can be invoked directly with `/<name>`, and can also be auto-invoked by Claude when relevant. A plugin **can absolutely contain skills**, and for new plugins Anthropic explicitly recommends the `skills/` layout over legacy `commands/`. The official docs also spell out the three-way distinction inside skills directories: a plain `SKILL.md` folder is a standalone skill, a folder with `.claude-plugin/plugin.json` is a plugin in the skills directory, and a `skills/<name>/SKILL.md` inside a plugin is a skill packaged within that plugin. ¹³

The practical rule is simple. A skill should remain local when it is **experimental, personal, project-specific, or still changing shape quickly**. Anthropic's own "when to use plugins vs standalone configuration" guidance says to start in `.claude/` for quick iteration and convert to a plugin when ready to share. A skill should be packaged into a plugin when you need **namespacing, versioning, distribution, discoverability, or multi-component behaviour around it**. The `claude-code-setup` plugin in Anthropic's official marketplace is a good example of a plugin that is mainly a skill, but whose value comes from being installable and reusable as a packaged setup aid. ¹⁴

Plugins and agents

Anthropic's plugin reference states that plugins can provide **specialised subagents** in `agents/`, and those agents can appear in `/agents`, be invoked manually, or be invoked automatically by Claude when appropriate. So yes: a plugin can contain agents, and official examples do so. `pr-review-toolkit` bundles six review agents; `feature-dev` bundles specialised agents behind a guided workflow; `code-review` in Anthropic's demo marketplace packages multiple agents for parallel PR

review. An agent solves **specialised delegated work**. A plugin solves **how that agent is shipped, versioned, discovered, and combined with other components**. ¹⁵

Plugins and hooks

Hooks are event-driven automation. Anthropic documents them as handlers for lifecycle events such as `SessionStart`, `PreToolUse`, `PostToolUse`, `SessionEnd`, and others, with hook actions including shell commands, HTTP calls, MCP tool invocations, prompts, and agentic verifiers. A hook solves **“this must happen on every matching event”**. A plugin lets you **bundle hooks with the skills, scripts, agents, configuration, and documentation they depend on**, then ship the whole set coherently. That is exactly what plugins like `hookify` and `security-guidance` illustrate. ¹⁶

Plugins and MCP

Anthropic’s comparison guide is explicit: **MCP connects Claude to external services**, while skills and plugins extend what Claude knows and how it behaves. A plugin can include or configure MCP servers through `.mcp.json` or inline manifest fields, and Anthropic’s official marketplace contains many plugins whose main value is “MCP, already configured for you”. So the right framing is: **MCP is the integration protocol; a plugin is one shipping format for MCP-backed workflows**. If all you need is a one-off connection to an external tool, MCP alone may be enough. If you need the connection plus team instructions, commands, agents, hooks, and managed rollout, then MCP belongs inside a plugin. ¹⁷

Building plugins in practice

Anthropic documents a plugin as a directory with optional manifest and default component locations. The manifest lives at `.claude-plugin/plugin.json`, and if present it defines metadata such as `name`, `displayName`, `version`, `description`, `author`, repository metadata, dependencies, component paths, default enablement, user configuration, and channels. If you include a manifest, `name` is the only required field, but for any serious plugin the manifest is effectively the stable contract for identity, namespacing, and versioning. Anthropic also documents a JSON Schema URL for editor autocomplete and validation. ¹⁸

The default directory layout documented by Anthropic is:

```
my-plugin/
├── .claude-plugin/
│   └── plugin.json
├── skills/
│   └── my-skill/
│       └── SKILL.md
├── agents/
├── hooks/
│   └── hooks.json
├── .mcp.json
├── .lsp.json
├── monitors/
│   └── monitors.json
```

```
├─ bin/
└─ settings.json
```

Anthropic repeatedly warns about one easy mistake: **only** `plugin.json` **belongs inside** `.claude-plugin/`. `skills/`, `commands/`, `agents/`, `hooks/`, and the rest must live at the plugin root. Anthropic also says `commands/` is the legacy flat-markdown command format and that new plugins should prefer the `skills/` structure. ¹⁹

A genuinely minimal plugin can be almost embarrassingly small:

```
my-first-plugin/
├─ .claude-plugin/
│   └─ plugin.json
└─ skills/
    └─ hello/
        └─ SKILL.md
```

That layout is straight from Anthropic's quickstart: a manifest for identity plus a skill folder whose `SKILL.md` carries frontmatter and instructions. Anthropic also documents `claude plugin init <name>` as the scaffolding path, which creates a skills-directory plugin under `~/.claude/skills/<name>/` for immediate local use. ²⁰

A more realistic team plugin looks more like this:

```
quality-review/
├─ .claude-plugin/
│   └─ plugin.json
├─ skills/
│   └─ code-review/
│       └─ SKILL.md
├─ agents/
│   └─ security-reviewer.md
├─ hooks/
│   └─ hooks.json
├─ .mcp.json
├─ scripts/
│   └─ validate.sh
├─ bin/
│   └─ review-helper
├─ settings.json
├─ README.md
└─ CHANGELOG.md
```

This matches Anthropic's documented "complex plugin" and "organise complex plugins" patterns, plus the explicit recommendation to include `README.md` when sharing and the recommendation to keep a `CHANGELOG.md` when using explicit versions. The official reference also shows a fuller example tree

with `scripts/`, `LICENSE`, `CHANGELOG.md`, `output-styles/`, and `themes/`, so documentation and release history are not garnish here; they are part of the maintainable shape. ²¹

A few practical structure details matter more than they first appear. Paths in manifest component fields must be **relative to the plugin root and start with `./`**; `skills` adds to the default `skills/` directory, whereas `commands`, `agents`, `outputStyles`, `experimental.themes`, and `experimental.monitors` replace the default locations unless you explicitly include the default path; and `settings.json` currently supports only the `agent` and `subagentStatusLine` keys, so plugins can change default main-agent behaviour when enabled. Anthropic also documents `defaultEnabled: false` for plugins that users should opt into, such as externally connected or higher-cost plugins. ²²

If your plugin needs persistent local state across updates, the reference's `${CLAUDE_PLUGIN_DATA}` guidance is particularly valuable. Anthropic documents a pattern where a `SessionStart` hook compares the bundled `package.json` against a copy in the plugin data directory and re-runs `npm install` only when dependencies changed. That is a very practical clue that **plugin upgrades are versioned cache swaps**, and any state or installed dependency you want to survive them should live under the plugin data directory rather than the versioned install root. ²³

Installation, publishing, and distribution

Anthropic documents installation in the current CLI as `claude plugin install <plugin>@<marketplace>` or the equivalent `/plugin install ...` flow from the interactive plugin manager. Installation supports **user**, **project**, **local**, and **managed** scopes. The official marketplace is `claude-plugins-official`, the community marketplace is `claude-community` after adding `anthropics/claude-plugins-community`, and Anthropic also maintains a demo marketplace from `anthropics/claude-code` for example plugins. ²⁴

A plugin marketplace is a **catalogue repo**, not magic. Anthropic's marketplace guide describes the workflow as: build the plugins, create `.claude-plugin/marketplace.json`, host that marketplace on GitHub or another git host, then let users add it with `/plugin marketplace add`. Marketplace entries can point to plugin code in the same repository via relative paths or to other repositories and registries via plugin sources. Anthropic explicitly recommends GitHub as the easiest host because you get version control, collaboration features, and straightforward sharing. ²⁵

For internal distribution, Anthropic provides two important mechanisms. The lighter-weight one is to add `extraKnownMarketplaces` to project `.claude/settings.json`, which prompts collaborators to install the organisation's marketplace after they trust the repo. The heavier-weight one is **managed settings**, where administrators can pre-register marketplaces, restrict allowed sources with `strictKnownMarketplaces`, and enable organisation defaults such as auto-updates. That is what turns plugins from "shared files in git" into something closer to controlled enterprise distribution. ²⁶

Publishing is not just "one install command away", and this is where your article can be genuinely useful. For **your own marketplace**, Anthropic's publishing model is basically Git-native: update plugin code and marketplace metadata, push changes, and users refresh or auto-update. For the **Anthropic community marketplace**, there is a review and submission pipeline: Anthropic says third-party submissions go through in-app forms, run through `claude plugin validate` and automated safety screening, and approved plugins are pinned to specific commit SHAs in the public community catalogue, which syncs nightly. Anthropic also states clearly that the official marketplace is curated separately and has no general application process. ²⁷

The trust model deserves explicit treatment. Anthropic's plugin installation docs warn that **plugins and marketplaces are highly trusted components that can execute arbitrary code on your machine with your user privileges**. Anthropic's official marketplace README adds the more specific warning that Anthropic does not control every MCP server, file, or software component inside a plugin and cannot guarantee that each one works as intended or will not change. So before installing a plugin, the sensible checklist is: review the source repository, read the homepage, understand what commands, hooks, MCP/LSP servers, and dependencies it will install, and prefer allowlisted internal marketplaces for broad team rollout. ²⁸

Versioning, updates, and maintainability

Anthropic's version management model is unusually explicit, and it is probably the most under-reported part of the plugin system. Claude Code uses the plugin's **resolved version as a cache key** to decide whether an update is available. Anthropic documents two main strategies. With an explicit `version` in `plugin.json`, users receive updates **only when that field changes**. If the plugin is git-backed and you omit `version`, Claude falls back to the **git commit SHA**, so every new commit becomes a new version. Anthropic explicitly recommends explicit versions for published plugins with stable release cycles and commit-SHA versioning for internal or rapidly iterating team plugins. ²⁹

That gives you a practical maturity model without turning the article into a maturity lecture. **Early internal plugins** can omit `version` and move fast with commit-SHA updates. **Stable shared plugins** should set explicit versions, use semantic versioning, keep a changelog, and release intentionally. Anthropic directly recommends semantic versioning for explicit versions and explicitly warns that pushing commits without bumping an explicit version does nothing for existing users. That warning is one of the most important maintenance facts in the whole system. ³⁰

Updates happen through `/plugin update`, marketplace auto-update, or startup auto-update when enabled. Official Anthropic marketplaces have auto-update enabled by default; third-party and local development marketplaces do not. Anthropic also documents that plugin updates during an active session require `/reload-plugins` for hooks, MCP servers, and LSP servers to switch to the new version path, while monitors require a session restart. This means a plugin maintainer cannot assume "I pushed a fix, therefore the running session is now using it." The operational behaviour is a bit more civilised than that, but only a bit. ³¹

For plugin dependencies, Anthropic's current system is more advanced than many people realise. Plugins can declare dependencies in `plugin.json` or marketplace entries; dependency version constraints can use semver ranges; and for git-backed sources Anthropic resolves those constraints against git tags using the `{plugin-name}-v{version}` convention. Anthropic also provides `claude plugin tag --push` to create compliant release tags. For teams building plugin ecosystems rather than one-off plugins, this is a major piece of maintainability infrastructure because it lets one plugin pin a tested range of another plugin instead of silently tracking "latest and good luck". ³²

Maintainability practices that are directly supported by the docs

A sound long-term operating model, grounded in official behaviour, looks like this:

- Validate in CI with `claude plugin validate --strict`, because Anthropic says unrecognised fields otherwise load with warnings rather than hard failures. ³³

- Decide **one versioning strategy per plugin** and document it. Do not set different versions in `plugin.json` and `marketplace.json` and hope for the best; Anthropic states that `plugin.json` wins silently. ³⁴
- Keep a `CHANGELOG.md` if you use explicit versions, because Anthropic recommends it directly. ³⁵
- Use `defaultEnabled: false` for plugins that carry cost, risk, or external-service side effects, so install does not automatically mean activate. ³⁶
- Prefer `${CLAUDE_PLUGIN_ROOT}` for bundled scripts and `${CLAUDE_PLUGIN_DATA}` for persistent state or dependencies that should survive updates. ³⁷
- Treat `README.md`, homepage, and plugin detail metadata as part of the product, because Anthropic's UI exposes what a plugin adds and the docs explicitly recommend including installation and usage guidance before sharing. ³⁸

Inferred best practice for real teams

The following points are not all spelled out by Anthropic as “best practice”, but they are strongly implied by the docs and supported by better community examples.

Separate operational ownership from authorship when you scale. Anthropic's marketplace model allows marketplace repos to point at relative paths, external repos, git subdirectories, and npm sources. That means larger organisations do not need one giant monorepo of plugins; they can run a curated catalogue that points to separately owned plugin repos. A community article from Michał Tuszyński explicitly argues for this separation so approval metadata and source code governance do not get tangled, which is good engineering judgement rather than official requirement. ³⁹

Use clear ownership files and review paths. Anthropic does not mandate CODEOWNERS in plugin repos, but the community marketplace `claude-market/marketplace` requires `CODEOWNERS`, `README.md`, and `LICENSE` for submitted plugins and formalises a review process. For internal teams, that is a sensible model: somebody must own the plugin, somebody must review behavioural changes, and somebody must own marketplace inclusion. ⁴⁰

Test behaviour, not just schemas. Anthropic's official tooling gives you validation and debugging, but not a complete behavioural test harness. So the practical pattern is to combine manifest validation, example usage, local install testing, `/reload-plugins`, and real-session scenarios before publishing widely. Anthropic's `plugin-dev` toolkit itself emphasises validation, testing, debug mode, and utility scripts, which is a strong hint about how the Anthropic team thinks plugin development should be done. ⁴¹

Design best practices and example plugins

Best practices from official docs and official Anthropic repos

The clearest official design lesson is to choose the **right abstraction boundary**. Anthropic's `Extend Claude Code` page lays out the extension stack so that you do not use plugins as a first reflex. Start with `CLAUDE.md` for always-on context, add skills for reusable workflows, add hooks for deterministic automations, add MCP for external systems, and **promote to a plugin when that bundle needs reuse across repos or users**. That is the opposite of building a “mega plugin” because it reminds you that packaging is justified by reuse, not by enthusiasm. ⁴²

Within a plugin, the official `plugin-dev` toolkit points towards several reusable practices: progressive disclosure in skills, strong trigger descriptions, clear responsibilities for agents, security-first hook design, portability through `${CLAUDE_PLUGIN_ROOT}`, relative paths, and validation/testing before deployment. Those are not only stated in prose; Anthropic's own demo and official plugins show the patterns in use. `commit-commands` is a compact command-focused plugin; `pr-review-toolkit` is a bundle of sharply scoped specialist agents; `hookify` is a plugin built around safe hook creation and management; `claude-md-management` focuses on a narrow documentation-maintenance workflow. None of them tries to be everything simultaneously, which is precisely the point. ⁴³

Anthropic's official examples also suggest a useful plugin taxonomy for your readers. Some plugins are **workflow bundles** such as `feature-dev` and `commit-commands`. Some are **specialist reviewer packs** such as `pr-review-toolkit` and `code-review`. Some are **integration wrappers** that package MCP or LSP configuration. Some are **meta-plugins** such as `plugin-dev` or `claude-code-setup` that help you build or choose other Claude Code customisations. That taxonomy is more helpful than discussing "plugins" as if they were all alike. ⁴⁴

Inferred best practice for plugin design

A few stronger recommendations emerge when you combine official docs with community experience.

Build around a workflow, not a technology. "A PostgreSQL MCP plugin" is rarely enough on its own. "Experiment triage", "incident review", or "feature release" are healthier plugin boundaries because they can cleanly combine skills, commands, agents, and integrations. This is also how the better official plugins are framed: `commit-commands` is not "git wrapper files"; it is a commit-and-PR workflow. `feature-dev` is not "some agents"; it is a structured feature-development process. ⁴⁵

Keep slash commands obvious and discoverable. Anthropic's official command-heavy plugins expose verbs that read like workflow entry points: `/commit`, `/commit-push-pr`, `/feature-dev`, `/revise-claude-md`, `/hookify`. Good commands announce intent. Bad ones become a private dialect. That sounds simple because it is simple, and simplicity is annoyingly effective. ⁴⁶

Use hooks carefully because they enforce behaviour. Anthropic's own comparison guide says hooks are for actions that must happen the same way every time and that guardrails belong in hooks because putting them in CLAUDE.md or a skill is only a request, not a guarantee. Community evidence reinforces the caution: the `skill-tools` plugin deliberately makes its auto-lint hook non-blocking and silently skips when dependencies are missing, and official marketplace issues show that shell-based hooks can have cross-platform and transcript-timing edge cases. So the best rule is to use hooks for deterministic guardrails and background checks, but to design them to fail safely and document their platform assumptions. ⁴⁷

Minimise unnecessary MCP dependencies and handle secrets as configuration, not prose.

Anthropic's manifest supports `userConfig` with `sensitive: true`, storing secrets in secure storage rather than `settings.json`, and exposes those values through substitutions for MCP/LSP configs, hooks, monitors, and subprocesses. That is a far better pattern than asking users to paste tokens into markdown. It also means plugins that depend on external services should usually install disabled or at least require explicit opt-in and documented setup steps. ⁴⁸

Avoid betting the farm on experimental components. Anthropic marks `themes` and `monitors` under the `experimental` key and explicitly warns that their schema may change while they stabilise. So if you want a broadly maintainable organisational plugin, build the core value on skills, agents,

hooks, and MCP first, and treat monitors/themes as optional extras rather than the essential centre.

49

Example plugins worth studying

`commit-commands` is a strong official example of a **narrow workflow plugin**. It packages a common git workflow behind a few memorable commands, documents what those commands do, and keeps the surface area intelligible. A data or ML team can learn from its discipline: do one boring, repeated thing well. 50

`pr-review-toolkit` is a strong official example of a **specialist-agent bundle**. It packages six narrowly scoped reviewers rather than one omnibus “review everything” agent. That is a healthy design pattern for code review, experiment review, or production incident review because it keeps responsibilities clear and allows parallelism. 51

`hookify` is a strong official example of a **meta-plugin**. Its value is not one workflow but a safer authoring experience for another Claude Code primitive, namely hooks. That is useful for your audience because it illustrates a second-order pattern: a plugin can package tooling that helps teams create and govern future customisation, not just run a task. 52

`skill-tools` is a useful community example because it shows two mature behaviours that official docs imply but do not drive home. First, it **pins** an external dependency in the hook for supply-chain caution. Secondly, the hook is designed to **degrade gracefully** and not block edits if its helper is unavailable. That is very good plugin citizenship. It remains a community repo, not official guidance, so it should be treated as instructive rather than canonical. 53

`claude-market/marketplace` is a useful community example of **governance around a marketplace**. Its contribution requirements include documentation, validation, required ownership metadata, and explicit quality standards. Again, this is not Anthropic policy, but it is one of the cleaner public examples of how to operationalise review and submission, especially for internal teams. 40

Plugin ideas for data science and ML teams

An **experiment review plugin** is a very good fit when the same review logic applies across multiple repos or experiment notebooks. It could package a command such as `/experiment-review`, one or more reviewer agents, a skill describing the team’s evaluation criteria, and optionally MCP integrations to experiment tracking systems or documentation sources. The maintenance risk is schema drift in experiment metadata and external-system permissions, so keep the review taxonomy stable and isolate system-specific logic in MCP-backed components. 54

An **ML code review plugin** is also a good fit if your team repeatedly wants checks around leakage, feature-store usage, evaluation protocol, offline/online symmetry, and packaging/deployment standards. The `pr-review-toolkit` model is the right pattern here: several narrowly defined reviewers rather than one “ML guru” agent with a god complex. This could be a plugin if it is reused across repos; if it is only relevant in one codebase, start as a local skill or agent first. 55

A **data pipeline review plugin** is a good fit when you need reusable checks around DAG design, idempotency, lineage assumptions, dbt model conventions, backfill safety, or SLA-aware change review. It probably wants agents, skills, and perhaps hooks, but it should use hooks carefully: a blocking hook

that fires after every edit is often excessive, while a manually invoked `/pipeline-review` or post-change reviewer is usually saner. ⁵⁶

A **feature engineering review plugin** is only a partial fit. If the “feature engineering playbook” is still changing weekly, keep it as local skills and maybe an agent. If it has hardened into repeatable review criteria with common commands and external catalogue dependencies, then promote it to a plugin. This is exactly the local-to-plugin transition Anthropic’s standalone-versus-plugin guidance suggests. ⁵⁷

A **model monitoring investigation plugin** is an excellent candidate for mature teams. It could combine a command such as `/investigate-drift`, an incident-analysis skill, one or more investigator agents, optional monitors for logs or status feeds, and MCP integrations to observability platforms. The caveat is operational maturity: if this plugin depends on monitors, remember that Anthropic still classifies monitor schema as experimental, so anchor the workflow in commands, skills, and integrations rather than relying entirely on monitor behaviour. ⁵⁸

A **release notes generation plugin** is often **not** the first thing that should become a plugin. If release notes are needed in one repo, a command or skill is enough. It becomes a plugin when the same release-note workflow, approval language, and source integrations are shared across many repos or teams. This is a good example to include in the newsletter because it teaches restraint: not every repeatable task deserves packaging. ⁴²

An **incident analysis plugin** is a strong fit for platform, data platform, and ML infra groups. It can combine a command, a root-cause-analysis skill, specialist agents, and MCP access to logs, tickets, and status systems. The maintenance burden lies in permissions, secrets, and external dependencies, so `defaultEnabled: false`, sensitive user config, and strong README documentation would be prudent. ⁵⁹

An **analytics QA or onboarding plugin** is a very good fit if the workflows span many similar repos. An onboarding plugin, in particular, captures the “team infrastructure” angle neatly: it can bundle setup skills, project commands, review agents, MCP suggestions, and repo-specific checks, much like Anthropic’s own `claude-code-setup` plugin recommends automations based on a codebase. ⁶⁰

Recommended newsletter angle and source map

The best angle

The best primary angle is **“From prompts to plugins: when Claude Code becomes team infrastructure.”** It works because it connects naturally to the posts you already have on memory layers, rules, hooks, skills, and agents; it uses Anthropic’s own extension stack to show why plugins are different; and it shifts the conversation away from “marketplace clicks” towards **packaging, standards, versioning, rollout, and ownership**. That is where the real value is for data scientists, ML engineers, and technical managers. ⁶¹

A strong secondary framing, which can serve as the article’s governing thesis or subtitle, is **“Claude Code plugins are not features; they are packaging.”** Anthropic’s official docs support that almost word-for-word in functional terms, and it is the cleanest way to disentangle plugins from skills, hooks, agents, and MCP. It is especially useful because many readers currently confuse “a skill in a repo” with “a plugin”, and this article can explain that the difference is not mystical; it is mostly about **unit of distribution and lifecycle**. ⁶²

A good follow-up article, but probably not the main one, would be **“How to avoid building a terrible Claude Code plugin.”** The research here gives plenty of material for it: over-broad plugins, brittle hooks, undocumented dependencies, no owner, vague commands, stale versions, and no update strategy. That topic is useful, but it is stronger **after** readers understand what plugins are for and why packaging is the missing layer. ⁶³

A source map you can trust

Official Anthropic documentation

- **Extend Claude Code** ⁴

Supports: the extension stack, plugin-as-packaging-layer framing, CLAUDE.md vs rules vs skills vs hooks vs MCP vs plugins, and context-loading comparisons.

Trust: **highest**.

Caveat: conceptual rather than operational.

- **Create plugins** ⁶⁴

Supports: quickstart, minimal plugin structure, complex plugin examples, sharing guidance, conversion from `.claude/` config, community submission flow.

Trust: **highest**.

Caveat: introductory; some advanced operational details live elsewhere.

- **Plugins reference** ⁶⁵

Supports: manifest schema, file locations, component behaviour, scopes, skills-directory plugins, user config, channels, validation, update semantics, persistent data, default enablement.

Trust: **highest**.

Caveat: dense and reference-like; easy for readers to miss the bigger story.

- **Discover and install prebuilt plugins** ⁶⁶

Supports: official/community/demo marketplaces, install scopes, auto-update, plugin manager UI, security warning, team marketplaces.

Trust: **highest**.

Caveat: user-facing emphasis more than maintainer-facing depth.

- **Create and distribute a plugin marketplace** ⁶⁷

Supports: marketplace schema, plugin sources, strict mode, GitHub hosting, private repos, seed directories, release channels, managed restrictions.

Trust: **highest**.

Caveat: still requires synthesis to become a maintainability story.

- **Constrain plugin dependency versions** ⁶⁸

Supports: dependency constraints, semver ranges, git-tag resolution, cross-marketplace dependency allowlists, `claude plugin tag --push`.

Trust: **highest**.

Caveat: advanced topic; many readers probably have not seen it.

Official Anthropic GitHub repositories

- `anthropics/claude-code` **plugin examples and demo marketplace** ⁶⁹

Supports: official examples of plugin families, plugin-dev toolkit themes, feature-dev structure,

demo marketplace packaging.

Trust: **very high**.

Caveat: example-oriented rather than policy-oriented.

- [anthropics/claude-plugins-official](#) **marketplace and plugin examples** ⁷⁰

Supports: official marketplace structure, internal vs external plugin split, trust warning, reference implementation, real examples such as [commit-commands](#), [pr-review-toolkit](#), [hookify](#), [claude-code-setup](#), [security-guidance](#).

Trust: **very high**.

Caveat: README-level guidance is secondary to the docs.

Community GitHub repositories

- [skill-tools/skill-tools-plugin](#) ⁵³

Supports: graceful degradation for hooks, dependency pinning, practical hook design.

Trust: **medium**.

Caveat: useful pattern, but not official and some install syntax may lag current docs.

- [claude-market/marketplace](#) ⁴⁰

Supports: community governance patterns such as contribution standards, required documentation, [CODEOWNERS](#), and validation.

Trust: **medium**.

Caveat: community policy, not Anthropic policy.

- [alirezarezvani/claude-skills](#) ⁷¹

Supports: ecosystem-scale maintenance signals such as changelog, contributing docs, conventions, releases, security files, and test coverage for scripts.

Trust: **medium**.

Caveat: large multi-tool ecosystem repo rather than a narrow “pure Claude Code plugin” example.

Community blog posts and opinion pieces

- **Pat Erichsen on custom marketplaces** ⁷²

Supports: marketplace-as-infrastructure framing, portable versioned sharing.

Trust: **medium-low**.

Caveat: useful conceptual framing, but some schema examples are not canonical Anthropic reference material.

- **Michał Tuszyński on enterprise plugin operations** ⁷³

Supports: organisational practices such as allowlists, seed directories, separate marketplace governance, and explicit review processes.

Trust: **medium-low**.

Caveat: opinionated and not official; treat as engineering judgement.

- **Tyler Folkman on skills versus plugins** ⁷⁴

Supports: pedagogical framing of plugins as platform extensibility and packaging.

Trust: **medium-low**.

Caveat: interpretative commentary, not source-of-truth documentation.

Open questions and limitations

I did **not** find official Anthropic documentation describing a formal plugin **signing**, provenance-attestation, or cryptographic trust model. The official trust controls I did find are source review, curated/community marketplace processes, managed allowlists, SHA pinning in catalogues, and general security warnings. So if you write about “trusted distribution”, it is safer to frame it around **catalogue curation and source governance**, not around guarantees that the ecosystem does not currently document. ⁷⁵

I also did **not** find a single official end-to-end guide for “long-term plugin product management”. The pieces exist, but they are distributed across the plugin guide, reference, marketplace guide, dependency-versioning guide, and official example repos. That fragmentation is part of why your proposed article could be genuinely useful: the knowledge is there, but it is not yet packaged for readers in one place. ⁷⁶

Finally, some community repositories still show older or alternate installation idioms. Where community guidance conflicts with or predates the current Anthropic docs, the Anthropic docs should win. In a piece meant to stay accurate, prefer official CLI syntax and current marketplace behaviour, then use community sources only for patterns, cautionary tales, and operational judgement. ⁷⁷

¹ ⁴ ¹⁰ ¹² ¹³ ¹⁷ ⁴² ⁴⁷ ⁵⁴ ⁵⁶ ⁶¹ ⁶² <https://code.claude.com/docs/en/features-overview>
<https://code.claude.com/docs/en/features-overview>

² ⁷ ⁸ ¹⁴ ¹⁸ ¹⁹ ²⁰ ²⁷ ³⁸ ⁵⁷ ⁶⁴ ⁷⁵ ⁷⁶ <https://code.claude.com/docs/en/plugins>
<https://code.claude.com/docs/en/plugins>

³ ⁵ ¹⁵ ¹⁶ ²¹ ²² ²³ ²⁴ ²⁹ ³⁰ ³¹ ³³ ³⁴ ³⁵ ³⁶ ³⁷ ⁴⁸ ⁴⁹ ⁵⁸ ⁵⁹ ⁶⁵ <https://code.claude.com/docs/en/plugins-reference>
<https://code.claude.com/docs/en/plugins-reference>

⁶ ⁹ ²⁵ ³⁹ ⁶⁷ <https://code.claude.com/docs/en/plugin-marketplaces>
<https://code.claude.com/docs/en/plugin-marketplaces>

¹¹ ²⁶ ²⁸ ⁶⁶ <https://code.claude.com/docs/en/discover-plugins>
<https://code.claude.com/docs/en/discover-plugins>

³² ⁶⁸ <https://code.claude.com/docs/en/plugin-dependencies>
<https://code.claude.com/docs/en/plugin-dependencies>

⁴⁰ <https://github.com/claude-market/marketplace>
<https://github.com/claude-market/marketplace>

⁴¹ ⁴³ ⁴⁴ ⁶³ <https://github.com/anthropics/claude-code/tree/main/plugins/plugin-dev>
<https://github.com/anthropics/claude-code/tree/main/plugins/plugin-dev>

⁴⁵ ⁴⁶ ⁵⁰ <https://github.com/anthropics/claude-plugins-official/tree/main/plugins/commit-commands>
<https://github.com/anthropics/claude-plugins-official/tree/main/plugins/commit-commands>

⁵¹ ⁵⁵ <https://github.com/anthropics/claude-plugins-official/tree/main/plugins/pr-review-toolkit>
<https://github.com/anthropics/claude-plugins-official/tree/main/plugins/pr-review-toolkit>

⁵² <https://github.com/anthropics/claude-plugins-official/tree/main/plugins/hookify>
<https://github.com/anthropics/claude-plugins-official/tree/main/plugins/hookify>

⁵³ ⁷⁷ <https://github.com/skill-tools/skill-tools-plugin>
<https://github.com/skill-tools/skill-tools-plugin>

- 60 <https://github.com/anthropics/claude-plugins-official/tree/main/plugins/claude-code-setup>
<https://github.com/anthropics/claude-plugins-official/tree/main/plugins/claude-code-setup>
- 69 <https://github.com/anthropics/claude-code/blob/main/plugins/README.md>
<https://github.com/anthropics/claude-code/blob/main/plugins/README.md>
- 70 <https://github.com/anthropics/claude-plugins-official>
<https://github.com/anthropics/claude-plugins-official>
- 71 <https://github.com/alirezarezvani/claude-skills>
<https://github.com/alirezarezvani/claude-skills>
- 72 <https://dev.to/paderich87/versioning-your-ai-workflow-with-a-custom-claude-code-marketplace-4298>
<https://dev.to/paderich87/versioning-your-ai-workflow-with-a-custom-claude-code-marketplace-4298>
- 73 <https://dev.to/michaeltuszynski/your-claude-plugin-marketplace-needs-more-than-a-git-repo-5631>
<https://dev.to/michaeltuszynski/your-claude-plugin-marketplace-needs-more-than-a-git-repo-5631>
- 74 <https://tylerfolkman.substack.com/p/the-complete-guide-to-claude-skills>
<https://tylerfolkman.substack.com/p/the-complete-guide-to-claude-skills>